Linear and binary search

Outline

The search problem

Linear search

Binary search

The search problem

Introduction: The search problem

- In this section we state precisely the problem of looking for a value in an array
- It is one of the most frequent operations performed by a computer, and the starting point of the two algorithms of the chapter
- We also fix what an algorithm must return when the value is **not** in the array

Looking for a value

- Looking for a value in a collection of data is one of the most frequent operations performed by a computer:
  - a name in a list of contacts;
  - a code in a table of products;
  - a word in a dictionary.

- In this chapter the collection is an array of natural numbers, and we present two algorithms that solve the problem:
  - **linear search**, which works on every array;
  - **binary search**, which works only on arrays sorted in non-decreasing order, but which is much faster than linear search on long arrays.

Definition 1: search problem

Given an array $a[1..n]$ of n natural numbers and a target natural number m, the ***search problem*** *consists in determining if m is one of the elements of the array, i.e., if there exists an index $k \in \{1, 2, \ldots, n\}$ such that*

$$a[k] = m.$$

An algorithm for the search problem returns such an index k if it exists, and the value -1 otherwise.

Why the value -1?

- The value -1 is never a valid index of the array $a[1..n]$
- Nor is it a valid index of a C++ array, whose indices start from 0
- For this reason it can be used to signal that the target value has **not been found**

Summary

The search problem:

- Input: an array $a[1..n]$ of natural numbers and a target value m
- Output: an index k with $a[k] = m$, if it exists
- Otherwise the value -1, which is never a valid index

Two algorithms:

- linear search, for every array
- binary search, for arrays sorted in non-decreasing order

Linear search

Introduction: Linear search

- In this section we present the simplest algorithm for the search problem: compare the target with the elements one after the other
- It works on **every** array, with no assumption on the order of the elements
- We run it on an example and count exactly how many comparisons it performs

The problem

Given an array $a[1..n]$ of n natural numbers
and a target natural number m,
we consider the problem of determining
if m is one of the elements of the array.

- The simplest idea is to compare the target m with the elements of the array one after the other, from the first to the last
- and to stop as soon as an element equal to m is found.

This is the pseudo-code of the Linear Search algorithm
that determines if a target value m is present
in an array $a[1..n]$ of n natural numbers.

Algorithm: LinearSearch

Input : An array $a[1..n]$ of n natural numbers and a target value m

Output : Index of m in a, or -1 if not found

```
1  function LinearSearch(a, n, m)
2      i := 1
3      while i ≤ n do
4          if a[i] = m then
5              return i          // Element found at index i
6          end
7          i ← i + 1
8      end
9      return −1                 // Element not found
10 end
```

Run in Colab: Python · C++ · Java

How LinearSearch works

- The algorithm scans the array sequentially from index 1 to n
- At each iteration, it compares the element $a[i]$ with the target m (line 4)
- If $a[i] = m$, the element is found and index i is returned (line 5)
- If the loop completes without a match, the element is not in the array and -1 is returned (line 9)

If m appears more than once in the array, the algorithm stops at the first match:
the returned index is the **smallest** index k such that $a[k] = m$.

Example (1/2)

Consider the array $a = [4, 2, 7, 1, 5, 9, 3]$ with $n = 7$.

Search for $m = 5$:

iteration	i	$a[i]$	action
1	1	4	$a[i] \neq m$
2	2	2	$a[i] \neq m$
3	3	7	$a[i] \neq m$
4	4	1	$a[i] \neq m$
5	5	5	$a[i] = m \Rightarrow$ Found!

Accordingly, element 5 is found at index 5.

Example (2/2)

Consider the array $a = [4, 2, 7, 1, 5, 9, 3]$ with $n = 7$. Search for $m = 6$:

iteration	i	$a[i]$	action
1	1	4	$a[i] \neq m$
2	2	2	$a[i] \neq m$
3	3	7	$a[i] \neq m$
4	4	1	$a[i] \neq m$
5	5	5	$a[i] \neq m$
6	6	9	$a[i] \neq m$
7	7	3	$a[i] \neq m$
–	8	–	$i > n \Rightarrow$ loop ends, Not found!

Accordingly, element 6 is not in the array (return -1).

Counting the comparisons

- The work done by linear search is measured by the number of comparisons $a[i] = m$ at line 4
- In the example:
  - 5 comparisons are needed to find $m = 5$;
  - all the $n = 7$ elements are compared to conclude that $m = 6$ is not in the array.
- The next observation gives the exact number of comparisons in general

Observation 1: number of comparisons of linear search

Given an array $a[1..n]$ of n natural numbers and a target value m:

- *if k is the smallest index such that $a[k] = m$, then `LinearSearch` performs exactly k comparisons at line 4 and returns k;*
- *if m is not in the array, then `LinearSearch` performs exactly n comparisons at line 4 and returns -1.*

Proof

The variable i takes the values $1, 2, 3, \ldots$ in this order, and at each value $i \leq n$ exactly one comparison $a[i] = m$ is performed.

- If k is the smallest index with $a[k] = m$, then $a[i] \neq m$ for $i \in \{1, 2, \ldots, k-1\}$, so the algorithm does not stop at these values of i; for $i = k$ the comparison is true and the algorithm returns k. The comparisons are those with $i = 1, 2, \ldots, k$, i.e., k comparisons.

- If m is not in the array, every comparison is false, so the loop is executed for $i = 1, 2, \ldots, n$ and stops when $i = n + 1$. The comparisons are n, and the algorithm returns -1 at line 9.

□

Strength and price of linear search

Linear search makes **no assumption**
on the order of the elements.

Its price is that, when the target is absent
(or it is in the last position),
all the n elements of the array must be examined.

Summary

Linear search:

- compares m with $a[1], a[2], \ldots$ in this order and stops at the first match
- works on every array and returns the **smallest** index k with $a[k] = m$

Number of comparisons:

- exactly k if the first occurrence of m is at index k
- exactly n if m is not in the array

Binary search

Introduction: Binary search

- In this section the array is **sorted**, and we exploit the order to discard half of the array at each step
- We write the pseudo-code of binary search, run it on an example and draw its search intervals
- Finally we bound its number of iterations: about $\log_2 n$ instead of n

Looking up a word in a dictionary

Think of looking up a word in a paper dictionary:
nobody reads it from the first page.

We open it in the middle and,
since the words are in alphabetical order,
we know in which half to continue.

Binary search applies the same idea
to a sorted array.

Definition 2: sorted array

An array $a[1..n]$ *of* n *natural numbers is* ***sorted in non-decreasing order*** *if*

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

Given an array $a[1..n]$ of n natural numbers
sorted in non-decreasing order and a target natural number m,
we consider the problem of determining
if m is one of the elements of the array.

The search interval

- Binary search keeps a portion $a[left..right]$ of the array, called the **search interval**, which is the only portion that may still contain m
- At the beginning the search interval is the whole array
- At each iteration the target m is compared with the element $a[mid]$ in the middle of the search interval:
  - if $a[mid] < m$, then, since the array is sorted, all the elements $a[left], \ldots, a[mid]$ are smaller than m, and the search continues in the right half $a[mid + 1..right]$;
  - if $a[mid] > m$, then all the elements $a[mid], \ldots, a[right]$ are greater than m, and the search continues in the left half $a[left..mid - 1]$.

This is the pseudo-code of the Binary Search algorithm
that determines if a target value m is present
in an array $a[1..n]$ of n natural numbers
sorted in non-decreasing order.

Algorithm: BinarySearch (1/2)

Input : An array $a[1..n]$ of n natural numbers sorted in non-decreasing order and a target value m

Output : Index of m in a, or -1 if not found

```
1 function BinarySearch(a, n, m)
2     left := 1                                   // Left boundary
3     right := n                                  // Right boundary
4     while left ≤ right do
5         mid := left + ⌊(right − left)/2⌋        // Middle index
6         if a[mid] = m then
7             return mid                          // Element found
8         end
```

Algorithm: BinarySearch (2/2)

```
function BinarySearch(a, n, m) (continued)
    while left ≤ right (continued) do
 9      if a[mid] < m then
10          left ← mid + 1          // Search in right half
11      end
12      else
13          right ← mid − 1         // Search in left half
14      end
15  end
16  return −1                       // Element not found
17 end
```

Run in Colab: Python · C++ · Java

How BinarySearch works (1/2)

- The algorithm maintains two indices, *left* and *right*, defining the search interval $a[left..right]$, which contains $right - left + 1$ elements
- At each iteration, it computes the middle index *mid* (line 5) and compares $a[mid]$ with the target m
- If $a[mid] = m$, the element is found and its index is returned (line 7)
- If $a[mid] < m$, the search continues in the right half by updating $left \leftarrow mid + 1$ (line 10)
- If $a[mid] > m$, the search continues in the left half by updating $right \leftarrow mid - 1$ (line 13)

How BinarySearch works (2/2)

- If the search interval becomes empty ($left > right$), the element is not in the array and -1 is returned (line 16)
- Since
$$0 \leq \lfloor (right - left)/2 \rfloor \leq right - left,$$
the index mid always lies in the search interval:
$$left \leq mid \leq right.$$

Example (1/4)

Consider the sorted array $a = [3, 9, 11, 12, 12, 16, 19]$ with $n = 7$.

Search for $m = 12$:

iteration	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] = m \Rightarrow$ Found!

Accordingly, element 12 is found at index 4.

Example (2/4)

Consider the sorted array $a = [3, 9, 11, 12, 12, 16, 19]$ with $n = 7$.

Search for $m = 10$:

iteration	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] > m \Rightarrow right \leftarrow 3$
2	1	3	2	9	$a[mid] < m \Rightarrow left \leftarrow 3$
3	3	3	3	11	$a[mid] > m \Rightarrow right \leftarrow 2$
–	3	2	–	–	$left > right \Rightarrow$ Not found!

Accordingly, element 10 is not in the array (return -1).

Example (3/4)

Consider the sorted array $a = [3, 9, 11, 12, 12, 16, 19]$ with $n = 7$.

Search for $m = 3$:

iteration	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] > m \Rightarrow right \leftarrow 3$
2	1	3	2	9	$a[mid] > m \Rightarrow right \leftarrow 1$
3	1	1	1	3	$a[mid] = m \Rightarrow$ Found!

Accordingly, element 3 is found at index 1.

Example (4/4)

Consider the sorted array $a = [3, 9, 11, 12, 12, 16, 19]$ with $n = 7$.

Search for $m = 19$:

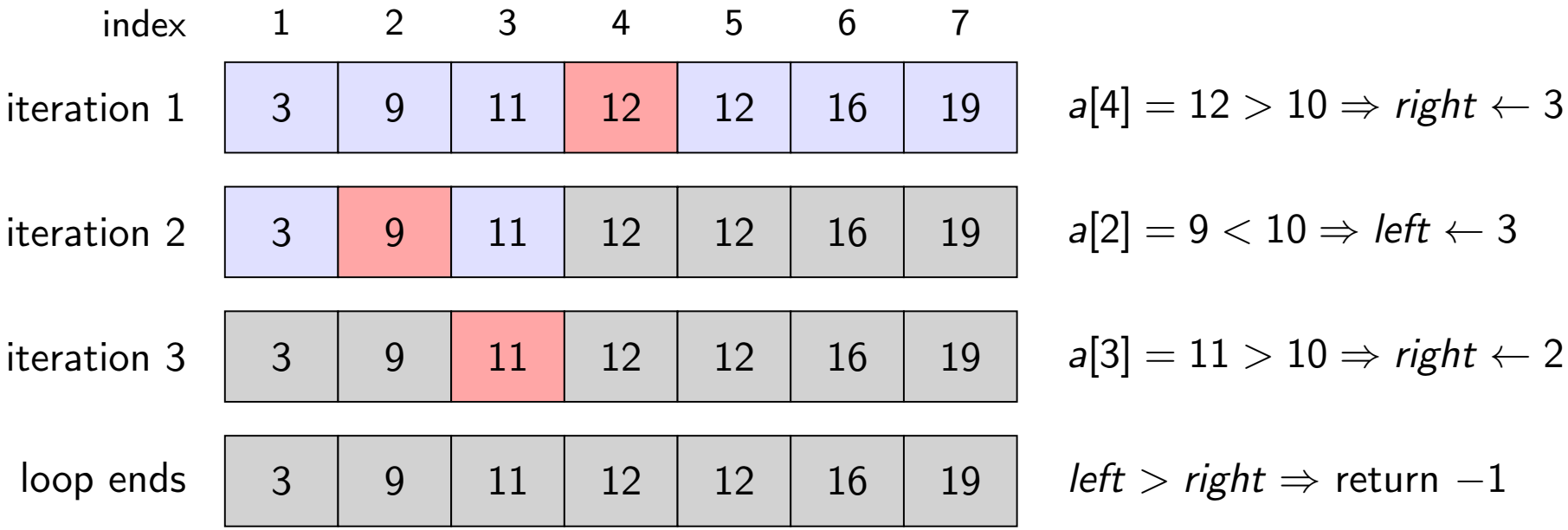
iteration	*left*	*right*	*mid*	$a[mid]$	action
1	1	7	4	12	$a[mid] < m \Rightarrow left \leftarrow 5$
2	5	7	6	16	$a[mid] < m \Rightarrow left \leftarrow 7$
3	7	7	7	19	$a[mid] = m \Rightarrow$ Found!

Accordingly, element 19 is found at index 7.

The search intervals

- The figure on the next slide shows the search interval at each iteration of the search for $m = 10$
- The elements outside the search interval (grey) are discarded
- The element $a[mid]$ compared with m is highlighted in red

The search intervals for $m = 10$



Search intervals of `BinarySearch` for $m = 10$ in $a = [3, 9, 11, 12, 12, 16, 19]$: blue cells form the search interval $a[left..right]$, the red cell is $a[mid]$, grey cells have been discarded.

Repeated values

- If m appears more than once in the array, binary search returns the index of **one** of its occurrences, not necessarily the first one

- For instance, for $a = [1, 2, 2, 2, 2, 2, 3]$ and $m = 2$:
  - the first iteration computes $mid = 1 + \lfloor 6/2 \rfloor = 4$;
  - since $a[4] = 2 = m$, the algorithm returns 4;
  - while `LinearSearch` returns 2.

Number of iterations

- Each iteration that does not find m discards the element $a[mid]$ and at least half of the remaining elements of the search interval
- This is why binary search is fast: the length of the search interval is (at least) **halved** at each iteration
- The next observation turns this idea into a bound on the number of iterations

Observation 2: number of iterations of binary search

Given an array $a[1..n]$ *of* $n \geq 1$ *natural numbers sorted in non-decreasing order and a target value* m*, the while loop of* `BinarySearch` *executes at most*

$$\lfloor \log_2 n \rfloor + 1$$

iterations.

- Approach of the proof: we show that the number of elements of the search interval is at least halved by every iteration that does not return

Proof (1/2)

Let $L = right - left + 1 \geq 1$ be the number of elements of the search interval at the beginning of an iteration that does not return. Since $mid = left + \lfloor (L-1)/2 \rfloor$, the new search interval has:

- $mid - left = \lfloor (L-1)/2 \rfloor$ elements, if $right \leftarrow mid - 1$;
- $right - mid = (L-1) - \lfloor (L-1)/2 \rfloor = \lceil (L-1)/2 \rceil$ elements, if $left \leftarrow mid + 1$.

In both cases the new number of elements is at most

$$\lceil (L-1)/2 \rceil = \lfloor L/2 \rfloor \leq L/2.$$

Proof (2/2)

At the beginning $L = n$. Hence, after k iterations that do not return, the search interval has at most $n/2^k$ elements.

An iteration is executed only if the search interval is not empty, so iteration $k + 1$ is executed only if

$$1 \leq \frac{n}{2^k}, \qquad \text{i.e.,} \qquad k \leq \log_2 n.$$

Since k is an integer, $k \leq \lfloor \log_2 n \rfloor$, and the number $k + 1$ of any executed iteration is at most $\lfloor \log_2 n \rfloor + 1$. □

The bound is attained

- In the example above, $n = 7$ and

$$\lfloor \log_2 7 \rfloor + 1 = 3$$

- The searches for $m = 10$, $m = 3$ and $m = 19$ execute exactly 3 iterations

- So the bound is **attained**

Linear search versus binary search

- The difference with linear search grows very quickly with n
- The table compares the number of comparisons of linear search when m is not in the array with the maximum number of iterations of binary search:

n	linear search n comparisons	binary search at most $\lfloor \log_2 n \rfloor + 1$ iterations
1 000	1 000	10
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

When to use binary search

- Binary search requires the array to be **sorted** in non-decreasing order
- On a sorted array of n elements it executes at most $\lfloor \log_2 n \rfloor + 1$ iterations, while linear search may need n comparisons
- If the array is not sorted, binary search may wrongly discard the half that contains m, and linear search must be used

Summary (1/2)

Binary search:

- works on arrays sorted in non-decreasing order
- keeps the search interval $a[left..right]$ and compares m with its middle element $a[mid]$
- continues in the right half if $a[mid] < m$, in the left half if $a[mid] > m$

Repeated values:

- it returns one of the occurrences of m, not necessarily the first one

Summary (2/2)

Number of iterations:

- every iteration that does not return at least halves the search interval
- at most $\lfloor \log_2 n \rfloor + 1$ iterations, and the bound can be attained

Comparison with linear search:

- 30 iterations instead of $1\,000\,000\,000$ comparisons for $n = 1\,000\,000\,000$
- on an unsorted array, only linear search is correct

Summary: key points of the chapter

- The search problem asks for an index k such that $a[k] = m$, or -1 if no such index exists
- Linear search works on every array, returns the first occurrence of m, and performs up to n comparisons
- Binary search works only on arrays sorted in non-decreasing order; it halves the search interval $a[left..right]$ at each iteration and executes at most $\lfloor \log_2 n \rfloor + 1$ iterations
- In the pseudo-code the indices go from 1 to n, in the Colab notebooks (Python, `C++`, Java) from 0 to $n - 1$: the returned indices differ by 1